Unleashing Corporate Intelligence with RAG

- By Dr. Anand Jayaraman

© 2025 TekFrameworks

Why Even the Best LLMs Need Help

- LLMs are powerful, but they:
  - Hallucinate facts
  - Have no memory of your organization's data
  - Are frozen in time (limited to training cutoff)
- Real-world questions often require external knowledge

© 2025 TekFrameworks

Retrieval Augmented Generation

RAG: Combine your documents with the LLM's language power

Instead of asking the model to “remember” everything,
we ask it to retrieve relevant information and then generate an answer.

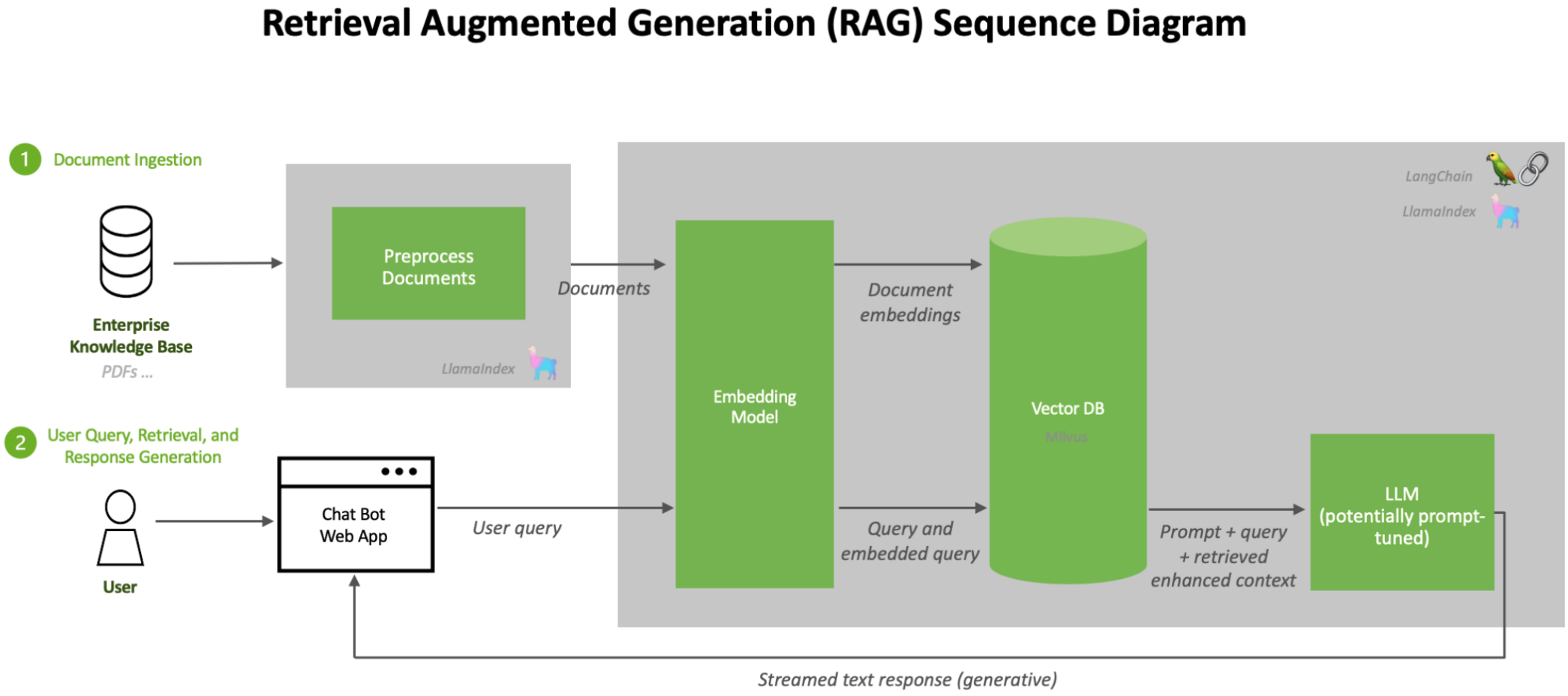
“RAG = Retrieval + Generation”
LLM gets facts from your docs before answering.

- Reduces hallucination
- Enables real-time access to private knowledge
- Can be combined with open or local models

Think of RAG as giving your LLM memory on demand

© 2025 TekFrameworks

RAG Architecture



https://developer.nvidia.com/blog/rag-101-demystifying-retrieval-augmented-generation-pipelines/

© 2025 TekFrameworks

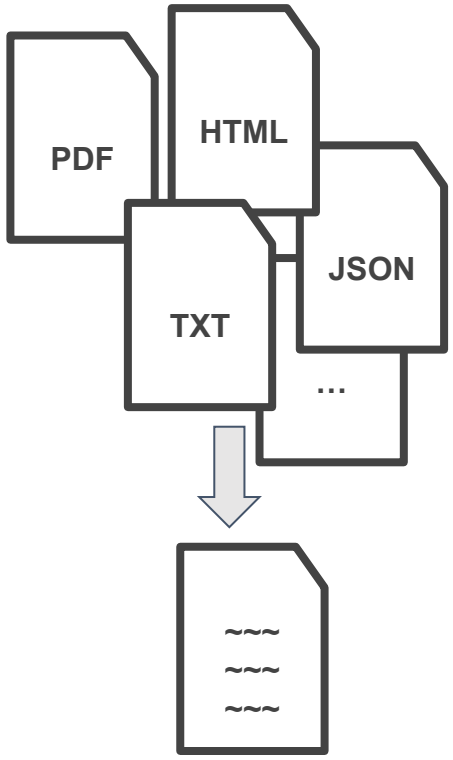
Retrieval Augmented Generation

TekFrameworks

- Load document (PDF, CSV, etc.)
- Split into chunks
- Generate vector embeddings
- Store in vector database (e.g., FAISS, Chroma)
- Retrieve top-k similar chunks based on query
- Pass retrieved content + query into LLM
- LLM generates grounded response

© 2025 TekFrameworks

Document Loaders



Arxiv
CSV
Discord
Email
EPub
EverNote
Facebook Chat
Figma
Git
GitHub
HTML
JSON
Markdown
Mastodon
MediaWiki Dump

Microsoft Word
MongoDB
Open Document Format (ODT)
Pandas DataFrame
PubMed
ReadTheDocs Documentation
Reddit
RSS Feeds
Slack
Snowflake
Telegram
X
URL
WhatsApp Chat
Wikipedia
XML
YouTube audio
YouTube transcripts

© 2025 TekFrameworks

Document Loaders

Loading a YouTube video transcript

- YoutubeLoader from LangChain **Community**
- loaders return a **list of Documents**

```
from langchain_community.document_loaders import YoutubeLoader
loader = YoutubeLoader.from_youtube_url("https://www.youtube.com/watch?v=8fEEbKJoNbU")
documents = loader.load()
```

Retrieval Augmented Generation

TekFrameworks

- Load document (PDF, CSV, etc.)
- Split into chunks
- Generate vector embeddings
- Store in vector database (e.g., FAISS, Chroma)
- Retrieve top-k similar chunks based on query
- Pass retrieved content + query into LLM
- LLM generates grounded response

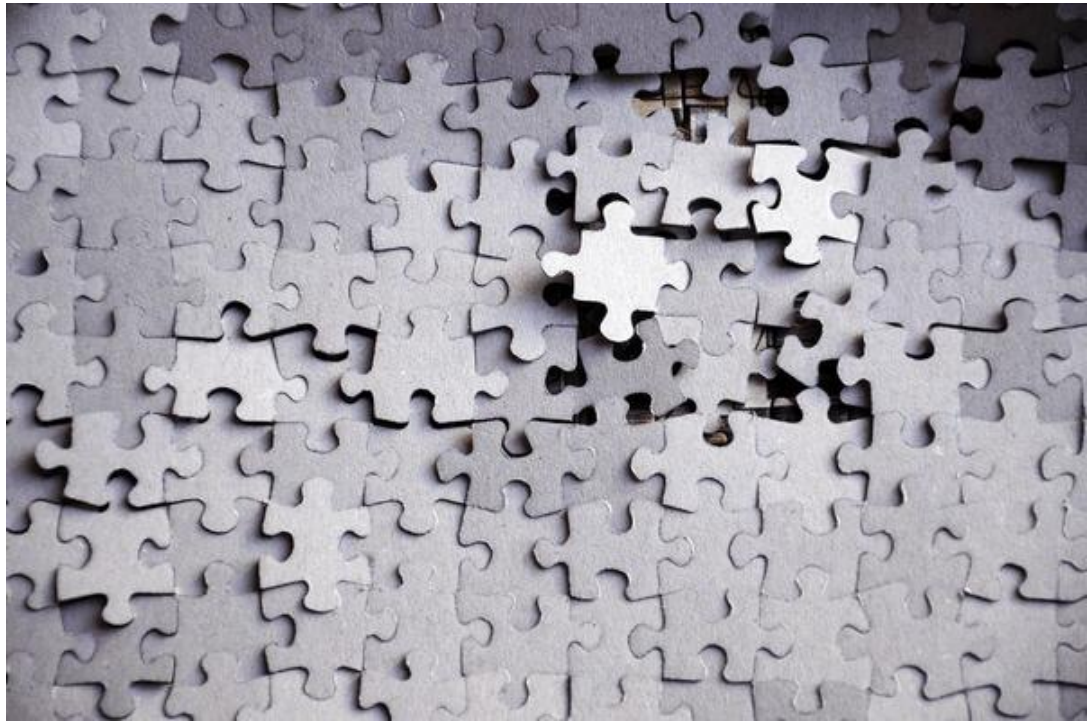
© 2025 TekFrameworks

Chunking

Chunking is the process of dividing large documents into smaller, manageable pieces (chunks) to facilitate efficient retrieval and accurate responses in RAG systems.

Why is Chunking Important?

- Enhances Retrieval Accuracy
- Maintains Contextual Integrity
- Optimizes Performance



Chunking is how we slice knowledge into pieces small enough for the model to 'remember' - but too small or too random, and meaning leaks out.

© 2025 TekFrameworks

Chunk Size

Larger chunks

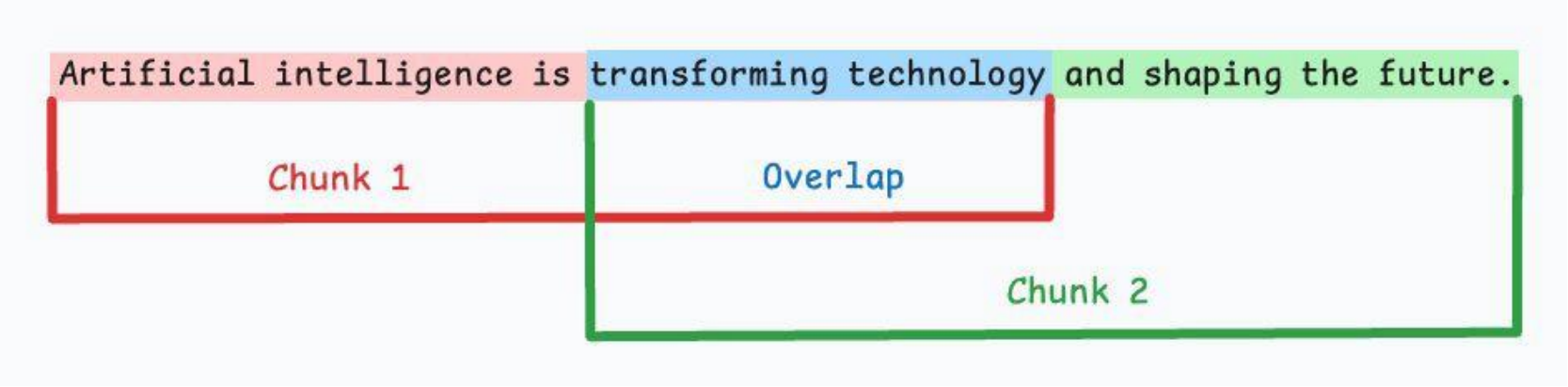
- More context
- Takes long to embed
- Retrieved chunk could have irrelevant information

Smaller chunks

- less context
- fast to embed
- compact relevant chunk retrieved

We look for sweet spot chunk size → 300-500 tokens without cutting off sentences

© 2025 TekFrameworks



Helps maintain context across chunks

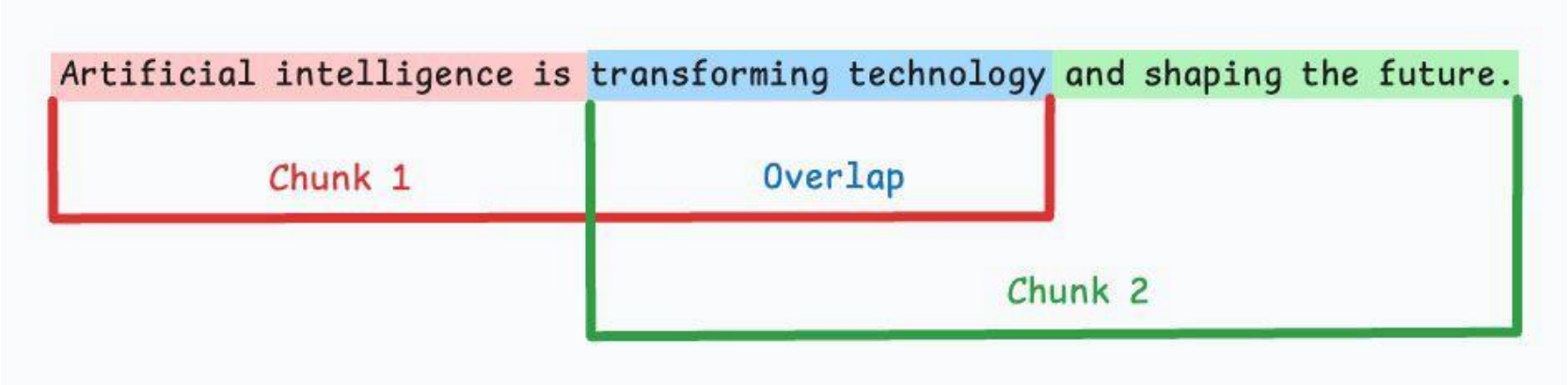
Sliding windowed chunking

© 2025 TekFrameworks

Fixed-size Chunking

TekFrameworks

- Simple implementation; may split sentences.

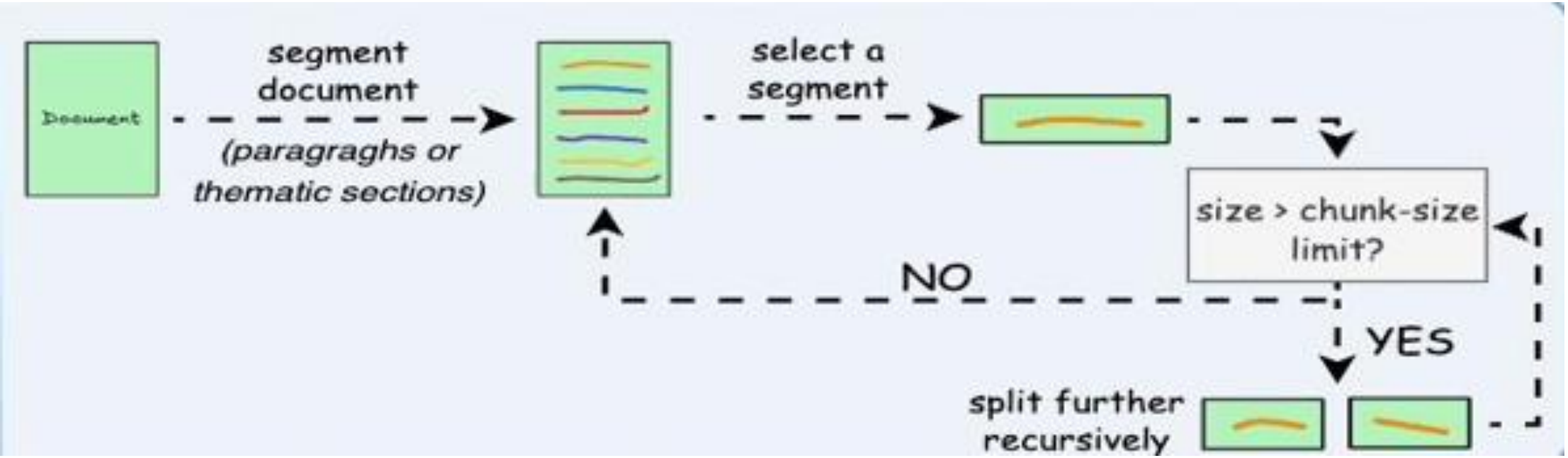


Sliding window helps maintain context across chunks

© 2025 TekFrameworks

Recursive Chunking

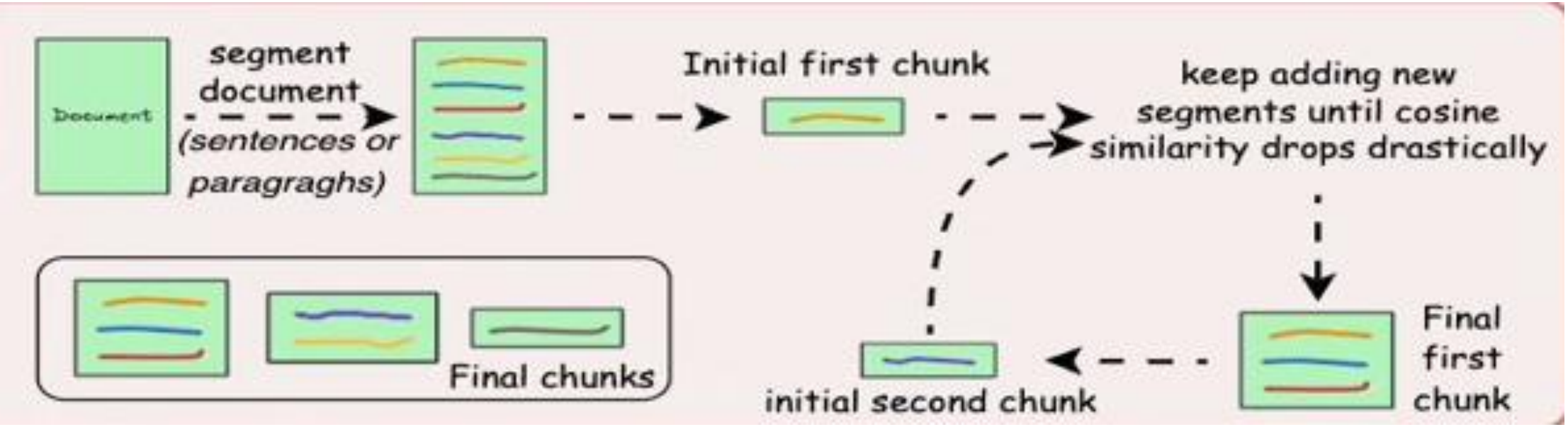
Respects sentence boundaries; maintains context.



© 2025 TekFrameworks

Semantic Chunking

Splits based on meaning; enhances retrieval relevance



© 2025 TekFrameworks

Text Splitters: 5 levels of text splitting

Characters / Tokens

Recursive Character

Document structure

Semantic Chunker

Agent-like Splitting

Chunking Demo

© 2025 TekFrameworks

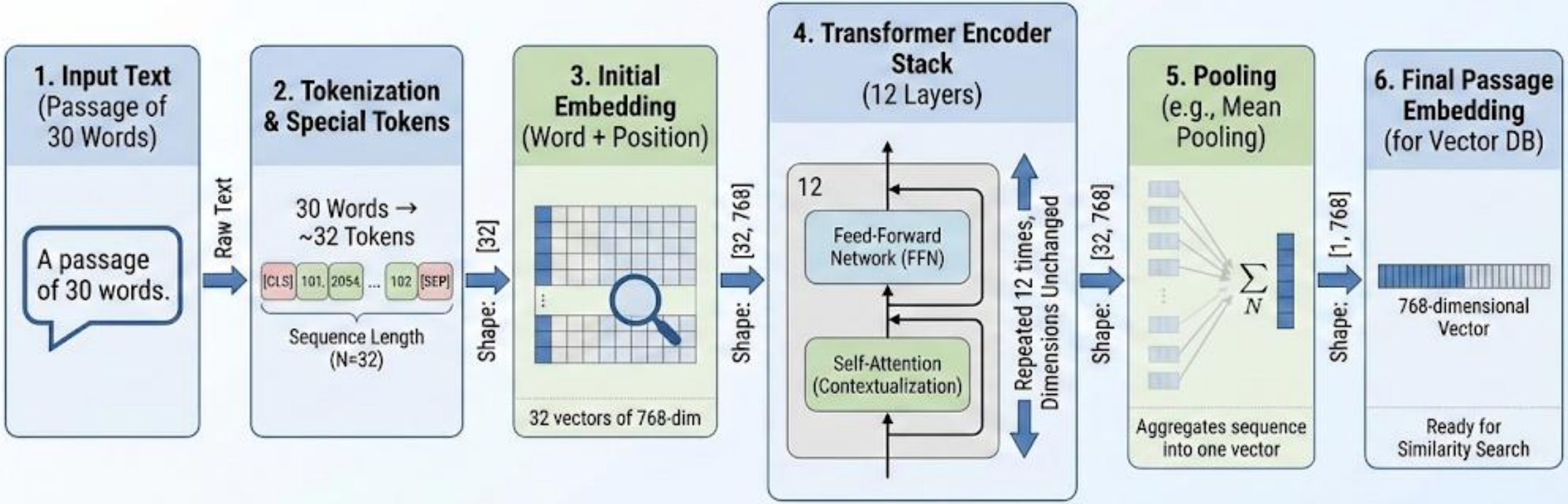
Retrieval Augmented Generation

TekFrameworks

- Load document (PDF, CSV, etc.)
- Split into chunks
- Generate vector embeddings
- Store in vector database (e.g., FAISS, Chroma)
- Retrieve top-k similar chunks based on query
- Pass retrieved content + query into LLM
- LLM generates grounded response

© 2025 TekFrameworks

Embedding Process: Passage to Vector Via BERT-Encoder



Dimensions Key: [Sequence Length, Hidden Dimension]
(e.g., [32, 768] = 32 tokens, each 768 dimensions)

© 2025 TekFrameworks

Embedding Models for RAG

TekFrameworks

Model	Dimensions	Speed	Multilingual	License	Best For
text-embedding-3-small	1536	Fast (via API)	Yes	OpenAI	Scalable apps using OpenAI stack
all-MiniLM-L6-v2	384	Very Fast	Yes	Apache 2.0	Lightweight, fast, easy to fine-tune
bge-base-en-v1.5 (BAAI)	768	Fast	No (English only)	MIT	High performance for local RAG
e5-base-v2 (MTEB winner)	768	Fast	Yes	Apache 2.0	Tasks requiring strong retrieval accuracy

- **MiniLM** is your go-to local embedding model for small-scale RAG — fast, well-tested, and available via Hugging Face.
- **BGE (BAAI General Embedding)** models are top-rated on MTEB benchmarks — great for precise document retrieval.
- **E5** (Embedding from Everything for Everything) models by Princeton are known for state-of-the-art retrieval across languages.

© 2025 TekFrameworks

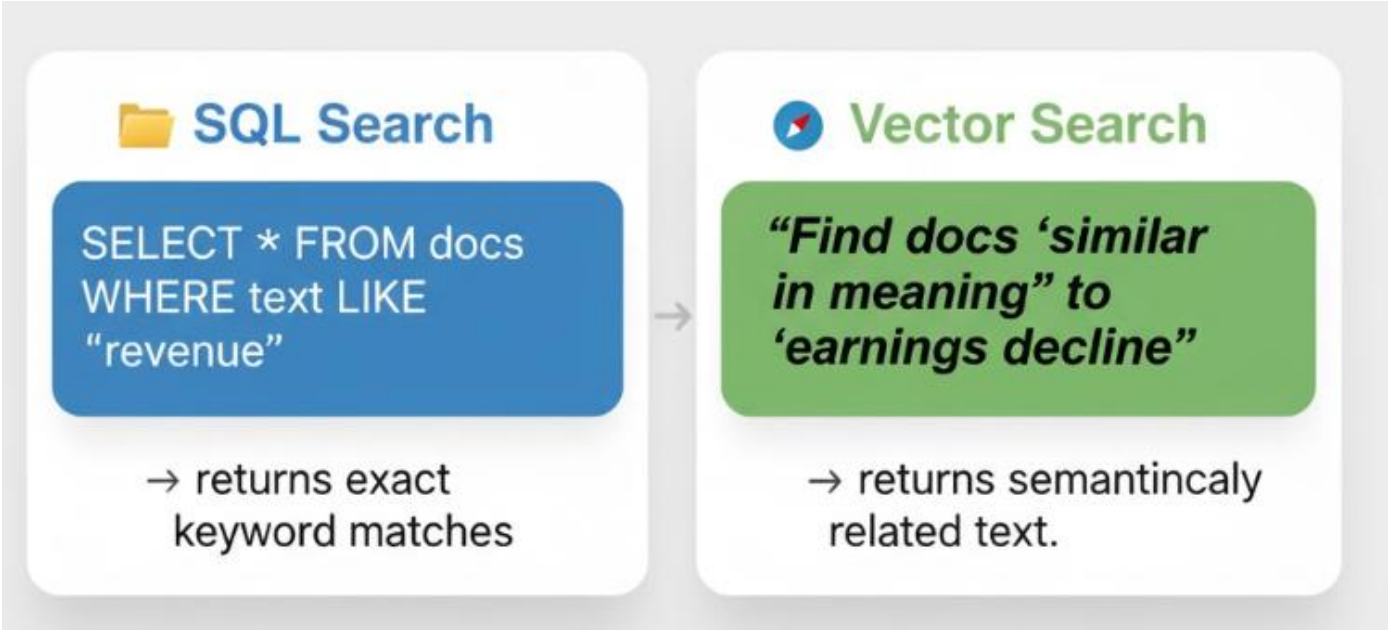
Retrieval Augmented Generation

TekFrameworks

- Load document (PDF, CSV, etc.)
- Split into chunks
- Generate vector embeddings
- Store in vector database (e.g., FAISS, Chroma)
- Retrieve top-k similar chunks based on query
- Pass retrieved content + query into LLM
- LLM generates grounded response

© 2025 TekFrameworks

Vector Databases: The Memory Layer of RAG



© 2025 TekFrameworks

Vector Databases

Feature	Why It Matters
Semantic Similarity Search	Finds documents *by meaning*, not just keywords — essential for natural-language queries.
High-Dimensional Embedding Storage	Stores each text chunk as a numerical vector (1k–3k dimensions) enabling quick semantic comparison.
Efficient Nearest-Neighbor Indexing	Enables millisecond retrieval of the most relevant chunks, even across millions of embeddings.
Hybrid Search (Keyword + Vector)	Combines structured filters (e.g., date, author) with semantic matching for precision.
Scalability & Latency Optimization	Handles billions of records and returns results fast — critical for production-scale RAG.
Ecosystem Integration	Connects seamlessly with frameworks like LangChain, LlamaIndex, and major LLM APIs.
Data Privacy & Governance Controls	Lets enterprises host vectors privately, ensuring compliance with data-residency policies.
Continuous Updating	Supports dynamic insertions / deletions as new knowledge is added without retraining the model.

© 2025 TekFrameworks

RAG & Tables

- Tables contain relationships and not sentences whereas LLMs are sentence learners

Product	Price	Stock
Widget A	$49.99	200
Widget B	$12.75	0

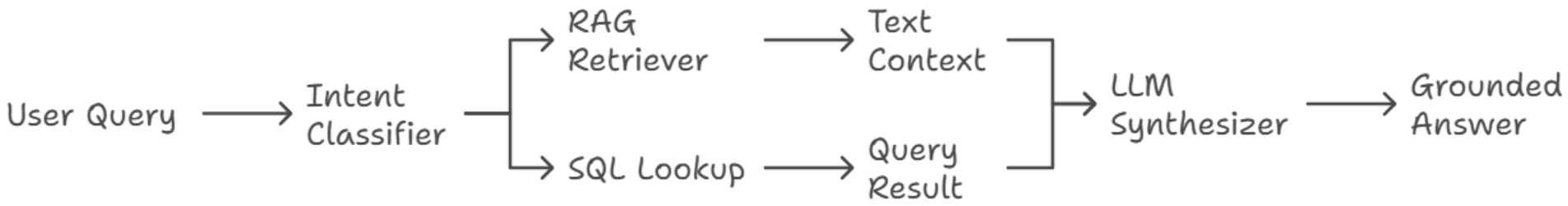
- On extraction this becomes

"Product Price Stock Widget A $49.99 200 Widget B $12.75 0 …"

- Tokenizers read linearly and not by columns. Column relations are lost.
- Serious enterprise RAG pipeline treats tables differently by converting them to structured context or off-loading them to a database.

© 2025 TekFrameworks

Handling Tables – Hybrid RAG Approach



- "Which products are out of stock and how do we handle refunds?"
  - RAG → fetch refund policy text; SQL → `SELECT product FROM inventory WHERE stock = 0`.

© 2025 TekFrameworks

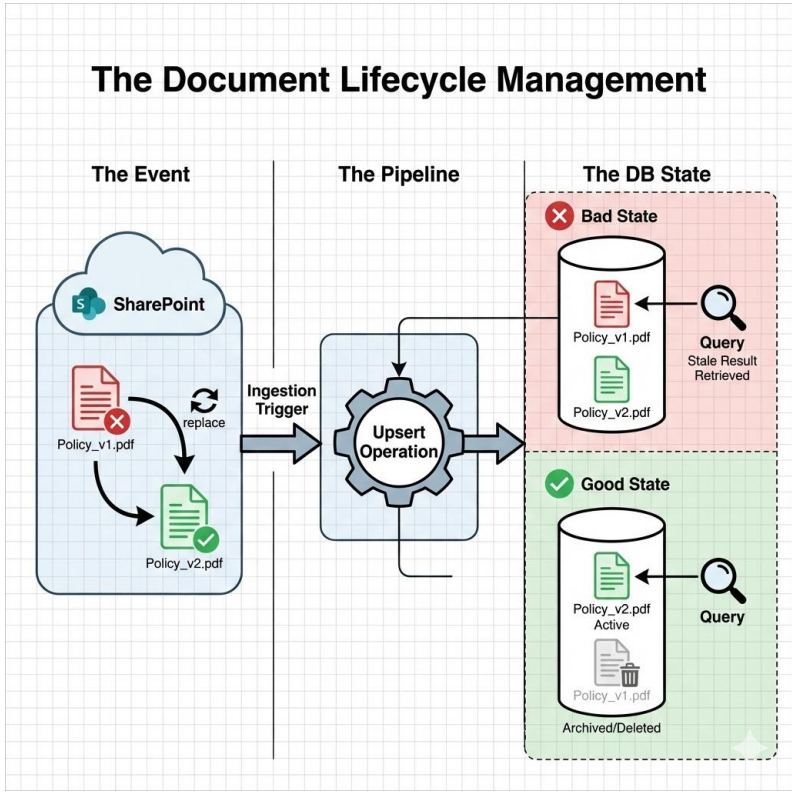
Real World Data - PDFs, Tables & Diagrams

- Scanned PDFs → require OCR + layout models (LayoutLMv3, DocFormer)
- Images/Diagrams → use Vision-Language models (BLIP, CLIP) → convert to captions or embeddings
- Tables → tokenizers flatten columns; consider CSV parsers → structured context or hybrid SQL lookup
- Charts & Forms → embed both text + numeric metadata
- Hybrid RAG = text RAG + structured retrieval (API / SQL)

© 2025 TekFrameworks

Handling "Stale" Data

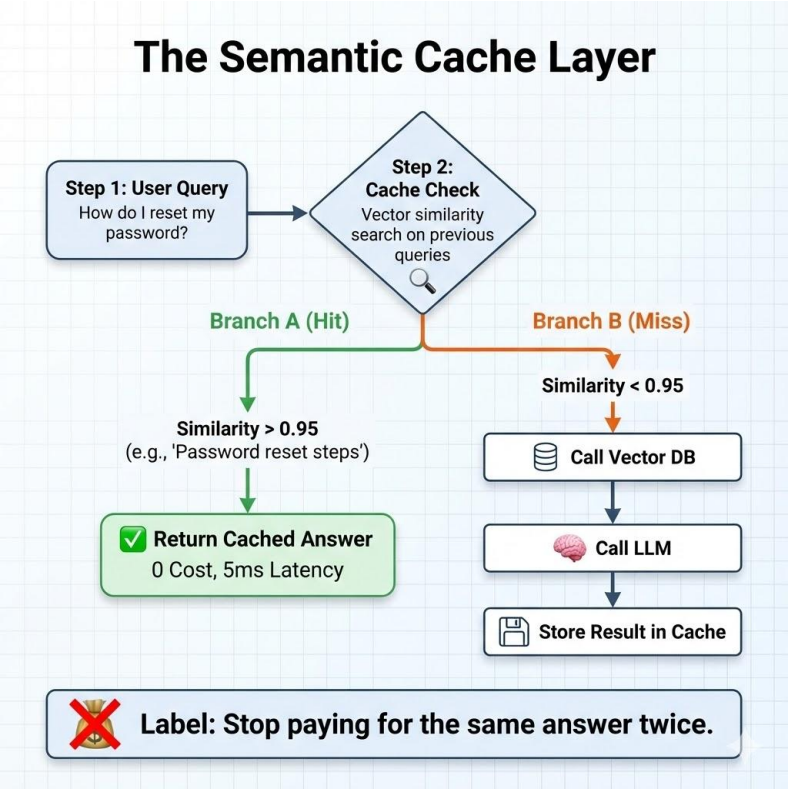
- **The “Tuesday Problem”**
  - HR updates “Remote work policy” on Monday. On Tuesday, a user asks about it. The RAG system retrieves the old policy because its still in index
- **Solution 1: Upsert Strategy**
  - Delete-then-insert: When a document is updated, you must identify all old chunks (via parent_doc_id) and delete them before adding new ones. Don’t just append!
- **Solution 2: Recency Weighting (Soft sorting)**
  - Don't just ingest text. Ingest Validity Dates
  - If you must keep old versions (for audit), use a **Decay Function**.
  - Boost the score of newer documents so they appear at the top of the context window



© 2025 TekFrameworks

Cost & Latency Strategy

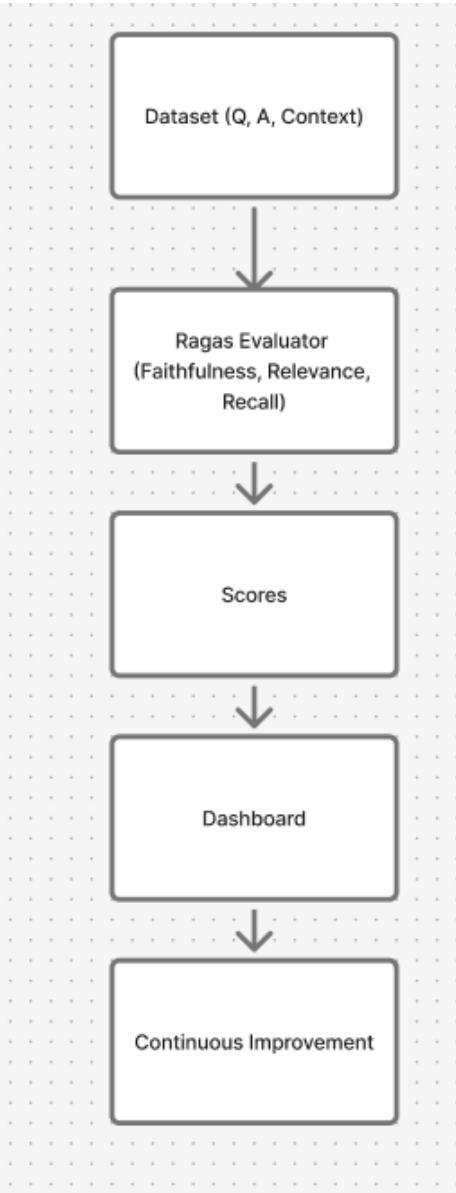
- Problem:
  - LLM inference is expensive and slow
  - In Enterprise (eg HR bots), 30-40% of queries are repetitive (eg: Holiday list, Reset password)
- Solution: Semantic Caching (Redis/GPTCache)
  - Vector Similarity on the query itself (not string matching)
  - If *Distance(New Query, Cached Query) < Threshold*, serve the cached LLM response
  - *Result:* Reduces cost and latency to near-zero for common questions



© 2025 TekFrameworks

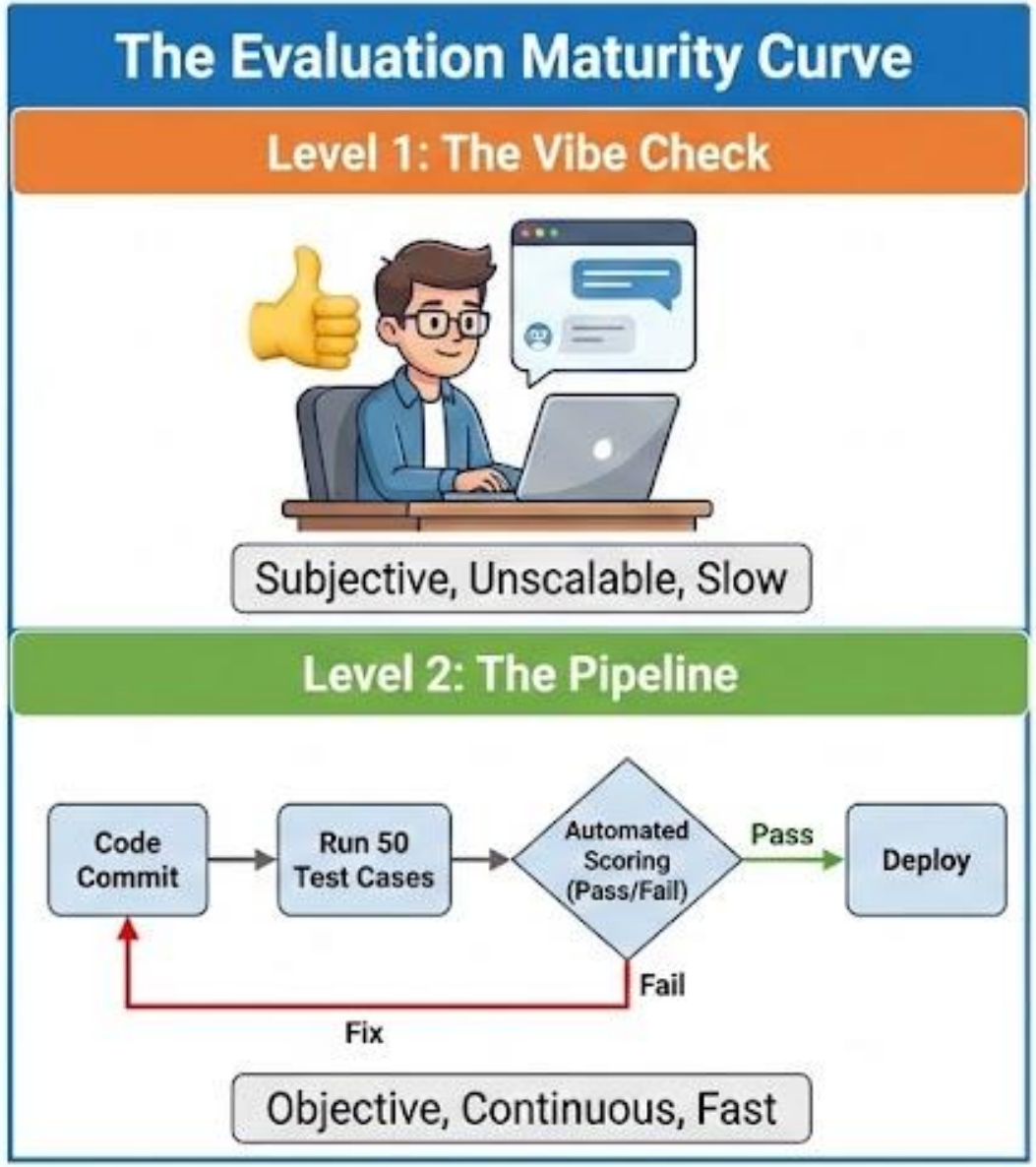
RAGAS – Retrieval Augmented Generation Assessment

- RAGAS – Open-source framework by Hugging Face for evaluation of RAG
- Compares the generated answer with both the question and retrieved context to check is response is grounded in evidence
- Applies Semantic similarity models for evaluation
- Metrics – Faithfulness, Context Precision, Recall and Relevance
- Used with LangChain LlamaIndex & TruLens to enable continuous evaluation and improvement in enterprise RAG pipelines



© 2025 TekFrameworks

Ops & Evaluation – The "Vibe Check" is Dead



The "Vibe Check" Trap:

- **Definition:** Manually asking the bot 5 questions ("What is the refund policy?") and saying, "Looks good to me!"
- **The Failure Mode:** It catches obvious errors but misses Regressions. (e.g., You fix the Refund answer, but accidentally break the Login answer).

The Solution: Automated Evaluation Pipelines

- **The "Golden Set":** You need a dataset of 50-100 QA pairs (Input Question + Ground Truth Answer) curated by humans1.
- **CI/CD Integration:** Every time an engineer pushes code (e.g., changes a prompt or chunk size), the system runs the Golden Set automatically.
- **Tools of the Trade: RAGAS / DeepEval:** Frameworks that output numerical scores for quality.

Thank You

We appreciate your time and look forward to your feedback on the session.

Contact us: TekFrameworks Consulting | info@tekFrameworks.com | www.tekFrameworks.com

Confidentiality Notice: This proposal is proprietary to **TekFrameworks Consulting Pvt. Ltd.** and is intended solely for the Client's evaluation. It may not be shared with third parties. The contents are non-binding; any engagement will be governed by a separate written agreement. For Client Use Only | TekFrameworks Consulting Pvt. Ltd.| © 2025 TekFrameworks